Indirect Computation of the DCT-IV via an External MDCT Backend: A Unified Mathematical Framework, GPU Adaptation, and Compliance Verification

Prof. Dr.Chris.P.BaconK[1]

[1]Institution.

Abstract

This paper proposes a unified framework for computing the DCT-IV through a three-stage pipeline—inverse folding, external MDCT backend, and post-processing—without implementing an MDCT core. A formal adaptation model is given for CPU, GPU, and hardware platforms. The contributions are threefold: (1) a rigorous proof of the folding equivalence theorem relating a 2N2N-point MDCT to an NN-point DCT-IV, from which a closed-form inverse folding map F-1F-1 is derived that involves only permutation, sign inversion, and zero-filling, with no arithmetic rounding; (2) the abstraction of backend adaptation into four hard preconditions H1-H4 (window-free, phase-matched, scalar scaling, natural ordering), together with decidable compliance criteria and a rejection condition; (3) a closed-form model—covering buffer pooling, stream partial order, throughput, latency, and error—for GPU environments, with numerical thresholds τ dtype τ dtype characterising the acceptability of floating-point backends. The whole is condensed into a single constrained system that formalises correctness, pipelinability, and verifiability as one object.

Keywords: DCT-IV, MDCT, folding equivalence, GPU adaptation, compliance verification, fixed-point error bounds

1 Abstract

This paper proposes a unified framework for computing the DCT-IV through a three-stage pipeline—inverse folding, external MDCT backend, and post-processing—without implementing an MDCT core. A formal adaptation model is given for CPU, GPU, and hardware platforms. The contributions are threefold: (1) a rigorous proof of the folding equivalence theorem relating a 2N2N-point MDCT to an NN-point DCT-IV, from which a closed-form inverse folding map F-1F-1 is derived that involves only permutation, sign inversion, and zero-filling, with no arithmetic rounding; (2) the abstraction of backend adaptation into four hard preconditions H1-H4 (window-free, phase-matched, scalar scaling, natural ordering), together with decidable compliance criteria and a rejection condition; (3) a closed-form model—covering buffer pooling, stream partial order, throughput, latency, and error—for GPU environments, with numerical thresholds τ dtype τ dtype characterising the acceptability of floating-point backends. The whole is condensed into a single constrained system that formalises correctness, pipelinability, and verifiability as one object. Keywords: DCT-IV; MDCT; folding equivalence; GPU adaptation; compliance verification; fixed-point error bounds

2 Introduction

The Discrete Cosine Transform of Type IV (DCT-IV) is a core component of the MDCT/IMDCT used in audio codecs such as MP3 and AAC. Two implementation routes exist: direct fast algorithms (FFT-based mapping or rotation-based recursive decomposition), and an indirect route that reuses an MDCT engine. The former requires a dedicated hardware or software core; the latter trades computational redundancy for module reuse, which is of considerable engineering value in resource-constrained embedded audio codecs. The indirect route, however, has long lacked a unified formal description: folding maps are scattered across specific implementations, backend IP adaptation relies on empirical tuning, and

GPU porting is often misconceived as RTL translation. The aim of this paper is to establish a complete chain from mathematical equivalence to engineering compliance, so that a "DCT-IV kit that does not implement an MDCT core" becomes a provable, verifiable, and cross-platform-deployable specification object.

3 Contributions:

1. A rigorous proof of the folding equivalence theorem and its inverse map (83); 2. Backend contract H1-H4 and decidable compliance criteria ($\mathrm{S4}, \mathrm{S8}$); 3. Closed-form buffer/stream/throughput/error models for GPU adaptation (§6 , §7 , §9); 4. A classification of vulnerabilities and their remedies (810); 5. A unified constrained system (§11).

4 Related Work

Direct fast DCT-IV algorithms: FFT-based mapping embeds the DCT-IV in an 8N8N-point FFT with pruning, achieving asymptotic complexity of about 1.79Nlog2N1.79Nlog2N. Givens-rotation-based recursive decomposition splits the transform kernel into N/4N/4 parallel $2 \times 22 \times 2$ rotations plus a 3- or 9-point module, yielding a regular structure well suited to hardware pipelines. The Britanak-Rao algorithm gives an optimal bound of 11 multiplications and 27 additions for the short block N=12N=12. Relationship between MDCT and DCT-IV: It is known that a 2N2N-point MDCT can be folded into an NN point DCT-IV, but this has mostly been used for theoretical analysis and has not been systematised into a deliverable kit. This paper engineers it into a three-stage pipeline and specifies the backend contract. GPU signal transforms: cuFFT provides batched FFT but has no native DCT-IV support; existing GPU DCT implementations are mostly direct algorithms and do not consider backend reuse or compliance verification. The GPU adaptation here emphasises buffer pooling and stream partial order rather than intra-kernel optimisation.

5 Mathematical Foundations

5.1 Notation

5.2 DCT-IV and MDCT definitions

Xb[k] = $\sum n = 0N-1ub[n]\cos[\pi N(n+12)(k+12)] \triangleq DN(ub)[k]Xb[k] = n = 0\sum N-1ub[n]\cos[N\pi(n+21)(k+21)] \triangleq DN(ub)[k]Yb[k] = \sum m = 02N-1xb[m]\cos[\pi N(m+12+N2)(k+12)] \triangleq M2N \rightarrow N(xb)[k]Yb[k] = m = 0\sum 2N-1xb[m]\cos[N\pi(m+21+2N)(k+21)] \triangleq M2N \rightarrow N(xb)[k]$

5.3 Folding operator

Partition xb ∈ R2Nxb ∈ R2N into four blocks of length N/2N/2, xb=(ab,bb,cb,db)xb=(ab,bb,cb,db), and define F(xb)=(-cbR-db,ab-bbR),vR[q]=v[N2-1-q]F(xb)=(-cbR-db,ab-bbR),vR[q]=v[2N-1-q]

5.4 Folding equivalence theorem

Theorem 1 (Folding equivalence) For any xb ∈ R2Nxb ∈ R2N, M2N → N(xb) = DN(F(xb))M2N → N(xb) = DN(F(xb)) Proof: Write $\theta m = \pi N(m+12+N2)(k+12)\theta m = N\pi(m+21+2N)(k+21)$. Using the symmetry of the cosine kernel, $\theta 2N-1-m = \theta m+\pi(k+12)\cdot 2 \Rightarrow \cos\theta 2N-1-m = (-1)?\cos\theta m\theta 2N-1-m = \theta m+\pi(k+21)\cdot 2 \Rightarrow \cos\theta 2N-1-m = (-1)?\cos\theta m$ More directly, split mm into [0,N/2)[0,N/2), [N/2,N)[N/2,N), [N,3N/2)[N,3N/2), [3N/2,2N)[3N/2,2N) and substitute m'=2N-1-mm'=2N-1-m in the third and fourth segments, using $\cos[\pi N(2N-1-m'+12+N2)(k+12)] = (-1)k\cos[\pi N(m'+12-N2)(k+12)]\cos[N\pi(2N-1-m'+21+2N)(k+21)] =$

$(-1)k\cos[N\pi(m'+21-2N)(k+21)]$ After symmetric folding the 2N2N-term sum reduces to NN terms, which is precisely DN(F(xb))[k]DN(F(xb)) [k].

5.5 Inverse folding map

Theorem 2 (Inverse folding) Let ub=(ub,L,ub,U)ub=(ub,L,ub,U), each of length N/2N/2. Define xb[m]= {ub[m+N2],0≤m<N20,N2≤m<3N2-ub[m-3N2],3N2≤m<2N(1)xb[m]= { } { ub[m+2N],0,-ub[m-23N],0≤m<2N 2N≤m<23N23N≤m<2N(1) Then F(xb)=ubF(xb)=ub, i.e. xb=F-1(ub)xb=F-1(ub), and F.F-1=IdF.F-1=Id. Proof: Substituting into the definition of FF: ab=ub,Uab=ub,U,bb=0bb=0,cb=0cb=0,db=-ub,Ldb=-ub,L hence F(xb)=(-0-(-ub,L),ub,U-0)=(ub,L,ub,U)=ubF(xb)=(-0-(-ub,L),ub,U-0)=(ub,L,ub,U)=ub. Corollary 1 There exists a sparse matrix $P \in \{0,\pm1\}2N \times NP \in \{0,\pm1\}2N \times N$ such that xb=Pubxb=Pub, with $\parallel P \parallel 2 = 1 \parallel P \parallel 2 = 1$; F-1F-1 involves no arithmetic rounding.

5.6 Complete chain

X=PB.MBIP.FB-1(U) (2) X=PB.MBIP.FB-1(U) (2) When the post-processing parameters satisfy S=1/ α S=1/ α and $\sigma\sigma$ matches the definition, X=DB(U)X=DB(U).

6 System Model

6.1 Three-stage pipeline architecture

text U $\rightarrow$ [FOLD] $\rightarrow$ β_x $\rightarrow$ [MDCT IP] $\rightarrow$ β_y $\rightarrow$ [POST] $\rightarrow$ X - FOLD: realises FB-1FB-1; permutation, sign inversion, and zero-filling only; - MDCT IP: external backend, realising MBIP=aMBMBIP=aMB; - POST: realises PBPB, comprising scaling, sign, saturation, and optional permutation.

6.2 Backend contract H1-H4

The kit interfaces only with backends satisfying the following four hard preconditions: Failure of any one $\rightarrow\rightarrow$ backend_reject; service is refused (§8.7).

6.3 Configuration validity

valid $\Leftrightarrow$ (Nmod2=0) $\wedge$ (N $\in$ Nip) $\wedge$ (Win $\geq$ Wu+1) $\wedge$ (Wout $\geq$ Wip) (3)valid $\Leftrightarrow$ (Nmod2=0) $\wedge$ (N $\in$ Nip) $\wedge$ (Win $\geq$ Wu+1) $\wedge$ (Wout $\geq$ Wip) (3)

References